



Presentazione Progetto SRS: Sistema di Logistica Droni Resiliente con AI Agentica

1. Definizione del Servizio e Dominio

Contesto e Dominio: Il progetto si inserisce nel settore dei trasporti e dei servizi critici per l'e-commerce, realizzando una piattaforma per la gestione di una flotta di droni per le consegne cittadine.

- **Stakeholder:** Gestori della flotta logistica, operatori di controllo volo e clienti finali (attesa ordini).
- **Workload:** Il sistema modella flussi continui di telemetria dai droni in volo (`drone_simulator.py`), la ricezione di nuovi ordini dai clienti (`client_simulator.py`), e le decisioni del server di smistamento centrale (`central_server.py`).
- **Aree di Rischio e Reliability:** Nel contesto "hardware volante", un guasto di rete o un'anomalia non gestita comportano rischi fisici severi, perdita del payload (costo diretto), e danni alla flotta. L'affidabilità si traduce nel far rientrare in sicurezza i droni o ri-pianificare tempestivamente in caso di condizioni ambientali e tecniche avverse.

2. Architettura Distribuita e Osservabilità

Il sistema è progettato come un'architettura a microservizi asincrona e "decoupled", costruita per la "graceful degradation".

- **Orchestrazione con Kubernetes (KIND):** L'infrastruttura è interamente containerizzata e governata tramite **KIND (Kubernetes IN Docker)**. Questa scelta strategica permette di eseguire un cluster Kubernetes locale aderente a standard di produzione. In questo specifico progetto, KIND gestisce:
 - *Auto-Healing e Reliability Base:* I componenti (Server, Broker Mosquitto, InfluxDB, Agenti e Simulatori) operano all'interno di Pod Kubernetes. Se durante il *stress test del professore* un processo va in crash, il ciclo di riconciliazione nativo di Kubernetes interviene riavviando il Pod quasi istantaneamente, migliorando l'RTO (Recovery

Time Objective).

- *Scalability Orientata al Dominio*: Attraverso i file YAML (`cluster.yaml`) deployati via `kubectl`), potrai dimostrare come sia facile "simulare il carico" aggiungendo parallelamente ulteriori repliche dei Pod dedicati ai droni. Kubernetes isola le risorse computazionali gestendo un traffico micro-segmentato tra l'hardware simulato (droni) e le componenti logiche.
- **Comunicazione Message-Driven (MQTT)**: L'utilizzo di Mosquitto (un servizio all'interno di cluster) come message broker garantisce che il server non debba interrogare costantemente i droni (polling), ma riceva asincronamente stream di telemetria. In caso di picchi (load spikes), la coda MQTT fa da cuscinetto assorbendo gli urti senza bloccare le comunicazioni centrali.
- **Osservabilità**: InfluxDB (`configs/influxdb.yaml`) viene impiegato per gestire le serie storiche dei dati di telemetria (stati batteria, variazioni di rotta). Questa base dati temporale è essenziale affinché il layer AI abbia contesto e storicità per le proprie decisioni.

3. Layer Agentico con Model Context Protocol (MCP)

In piena aderenza alla consegna, è stato sviluppato un livello agentico sofisticato basato su MCP, rispettando un paradigma strettamente *Domain-First*. I tool esposti (`drone_mcp_layer.py` , `mcp_server.py`) operano esclusivamente sul dominio delle consegne (modifica rotte, analisi guasti sui droni) e mai sull'infrastruttura backend.

- **Coordinamento e Separazione dei Ruoli**: Il flusso AI non è monolitico, bensì governato da un `agent_coordinator.py` che dirige moduli specializzati:
 - `health_agent.py` : Dedito all'analisi telemetrica, all'identificazione di modelli di degrado della batteria o anomalie motorie.
 - `logistic_agent.py` / `logistic_ai_brain.py` : Dedito alla ricalibratura del payload e alla riconsegna in caso di blocco di un drone in volo.
- **Perché l'Agentic AI?**: Un semplice sistema "if battery < 10% then land" è troppo rigido per scenari reali. L'AI permette di unire dati sul meteo, urgenza dell'ordine e storico delle batterie per suggerire una policy di atterraggio d'emergenza o un avanzamento a velocità ridotta.

4. Policy Operativa di Sicurezza (Guardrails e Human-in-the-Loop)

Il cuore della resilienza affidata all'AI è non fidarsi ciecamente dell'AI. Il sistema rispetta il *Principle of Least Privilege* e le direttive di *Operational Safety*:

- **Bounded Remediation e Interazione Umana:** Azioni considerate "sicure" (come rallentare la velocità per risparmio energetico) possono essere automatizzate. Azioni ad **alto impatto** (come ordinare l'abbattimento/atterraggio di emergenza in un'area pubblica, o annullare ordini massivi) passano tassativamente da `human_approval_manager.py`. Le richieste pendenti finiscono su `data/pending_approvals.jsonl`.
- **Limitazione delle Allucinazioni:** MCP restringe i tool a valle. L'agente non ha la capacità tecnica di spegnere i database, poiché i tool non esistono dal suo punto di vista.
- **Auditabilità:** Ogni interpretazione di telemetria e ogni richiesta (sia essa automatica o fermata per approvazione) viene loggata in `audit_actions.jsonl`. È sempre possibile ricostruire "perché l'agente ha consigliato questo blocco".

5. ROA (Return on Agent) e Considerazioni sui Costi

- L'introduzione dell'AI agentica previene la distruzione dell'hardware tramite il riconoscimento precoce di schemi di guasto (risparmio di CAPEX).
- Evita falsi positivi riducendo l'onere sull'analista umano (operator fatigue).
- **Cost Bound:** L'AI non analizza l'intero pacchetto MQTT costantemente, ma viene attivata (event-driven o batch-driven) solo in prossimità di condizioni anomale riportate dall'infrastruttura di health-checking deterministica, calmierando drasticamente i costi di token passati al LLM. Non appena l'AI va in dubbio, l'escalation avviene al livello umano, limitando i loop infiniti e i costi associati.